2026

# EHA361- MILESTONE 2

**SINETHEMBA KHUMALO**
**601571**

## TABLE OF CONTENTS

Introduction ..... 3

Buffer Overflow ..... 4

Buffer Overflow Attack ..... 4

Types Of Buffer Overflow Attacks ..... 4

Prerequisites ..... 5

Creating The Buffer Overflow Program ..... 6

Creating the C program file ..... 7

Code ..... 8-9

Compiling The Program .....9

Test Normal Operation .....10

Triggering The Buffer Overflow .....11

How the buffer overflow works ..... 12

Buffer overflow diagram ..... 12

Creating The One-Click Script ..... 13-14

How To Prevent Buffer Overflows ..... 15

Conclusion ..... 16

Reference List ..... 17

## INTRODUCTION

Milestone 2 demonstrates a simple buffer overflow program written in C that causes a stack-based buffer overflow leading to memory corruption and the vulnerabilities that can be exploited by attackers in the corrupted system.

Figure 1: Buffer Overflow Illustration

Source: Aaron Yoo (2021)

## BUFFER OVERFLOW

Buffers are memory storage regions that temporarily hold data while it is being transferred from one location to another. A buffer overflow occurs when the volume of data exceeds the storage capacity of the memory buffer and the program attempting to write the data to the buffer overwrites adjacent memory locations.

## BUFFER OVERFLOW ATTACK

Attackers exploit buffer overflow issues by overwriting the memory of an application. An attacker knowing the memory layout of a program can intentionally feed input that the buffer cannot store, and overwrite areas that hold executable code, replacing it with their own code triggering a response that damages files or exposes private information.

## TYPES OF BUFFER OVERFLOW ATTACKS

1. **Stack-based buffer overflows:** An attacker sends data containing malicious code to an application, which stores the data in a stack buffer. This overwrites the data on the stack, including its return pointer, which hands control of transfers to the attacker.
2. **Heap-based buffer overflows:** An attacker floods a program's memory space beyond the memory it uses for current runtime operations.
3. **Format string attack:** An attacker executes code that reads data in the stack, or cause segmentation faults in the application because an application processes input data as a command or does not validate input data effectively threatening the security and stability of the system.

## PREREQUISITES

1. **Kali linux virtual machine:** Terminal ready for the buffer overflow attack creation and execution

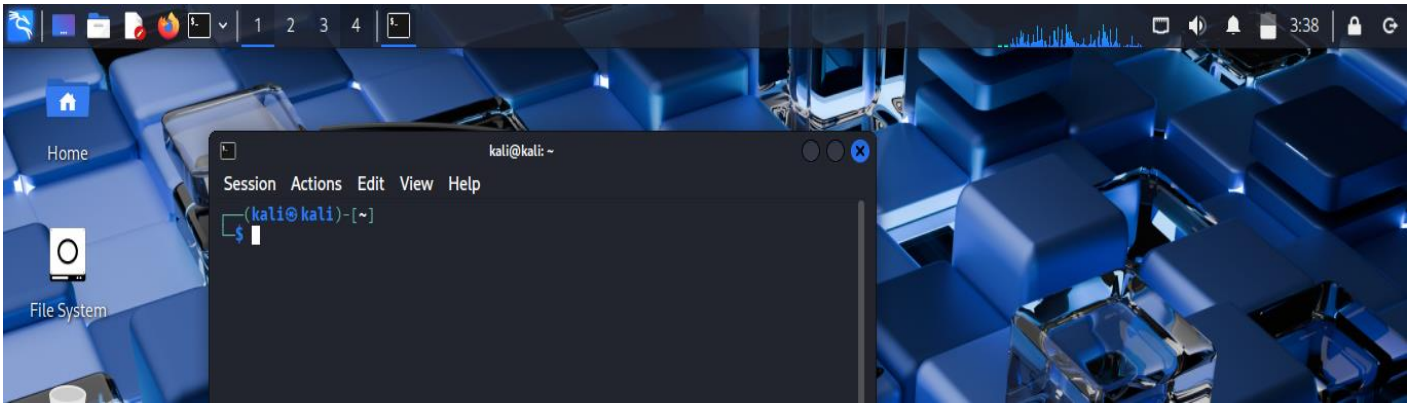


Figure 2: Isolated environment- kali linux virtual machine

2. **Installing/Verifying GCC in kali linux:** verifying if gcc is installed in my kali linux virtual machine before continuing creating my attack

Command used:

gcc –version

Results: gcc (Debian 14.3.0-5) 14.3.0

Figure 3: gcc version installed

## CREATING THE BUFFER OVERFLOW PROGRAM

**STEP 1:** Creating a new directory for the buffer overflow program

Command used: mkdir MyBufferOverflow

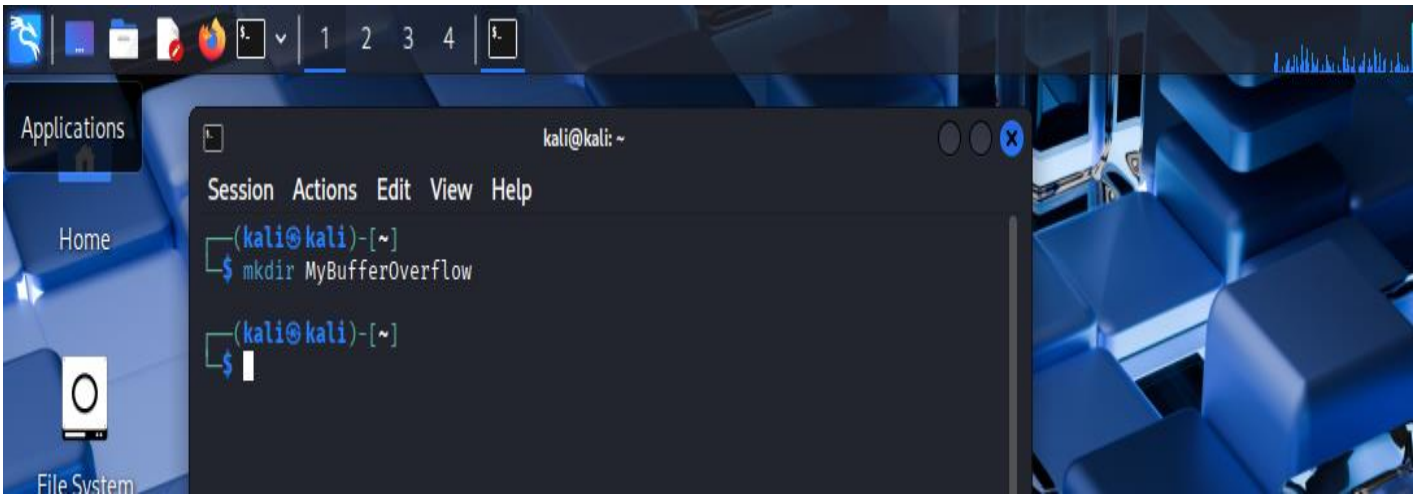


Figure 4: program directory

**STEP 2:** Changing directory to the new created directory MyBufferOverflow

Command used:

cd MyBufferOverflow

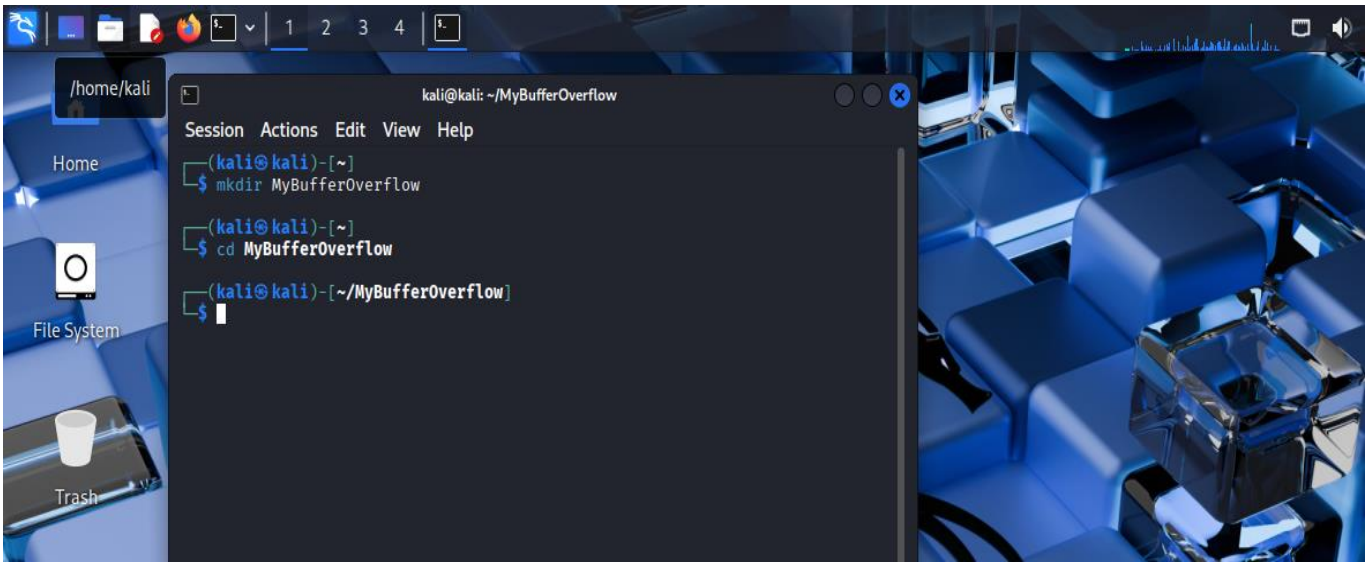


Figure 5: Navigated to new directory MyBufferOverflow

**STEP 3:** Creating the C program file

Command used:

nano TheBufferOverflow.c

**Step 4:** Writing the code for the buffer overflow, the code indicates that the program buffers 16 characters, copies input without checking size and shows the normal state versus the buffer overflow state.

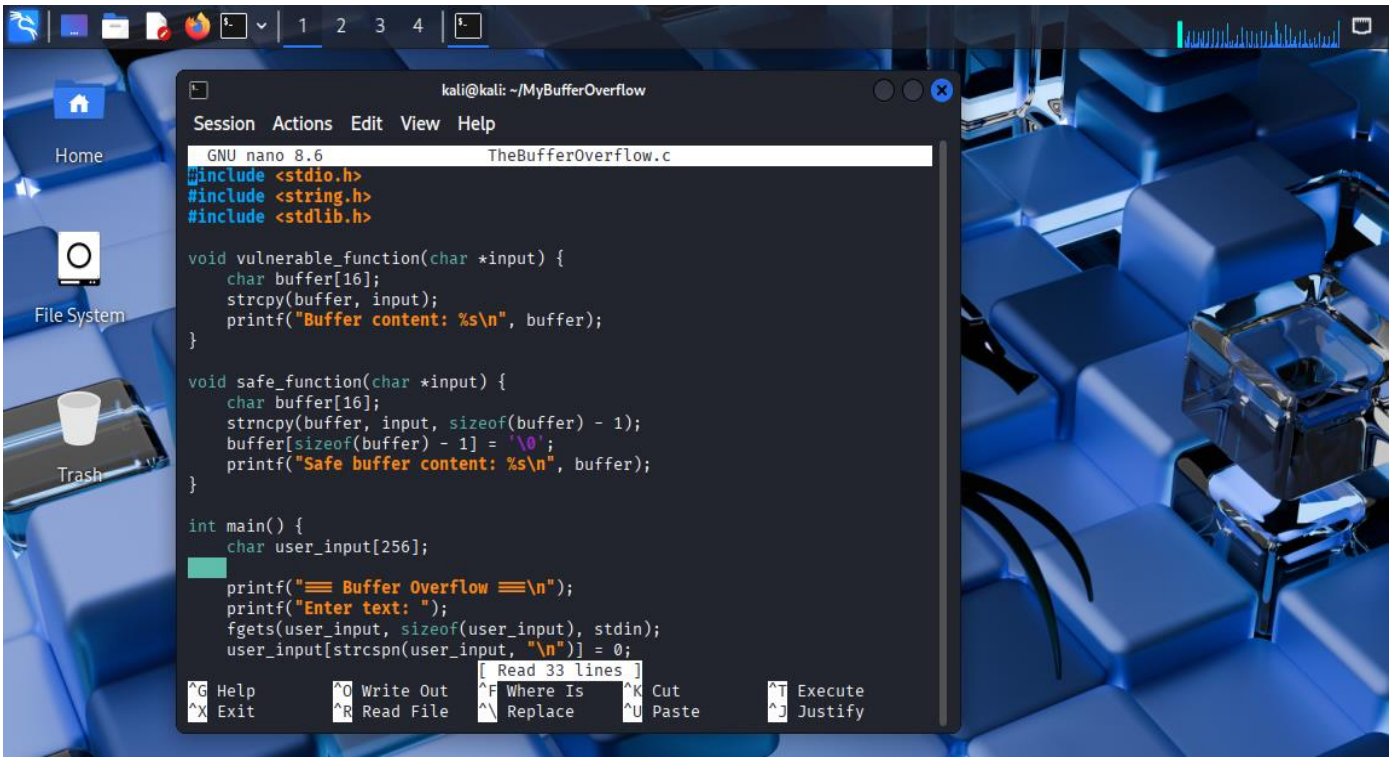


Figure 6: nano

TheBufferOverflow.c file successfully created

Figure 7: TheBufferOverflow.c file

**FULL CODE**

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>


void vulnerable_function(char *input) {

    char buffer[16];

    strcpy(buffer, input);

    printf("Buffer content: %s\n", buffer);

}


void safe_function(char *input) {

    char buffer[16];

    strncpy(buffer, input, sizeof(buffer) - 1);

    buffer[sizeof(buffer) - 1] = '\0';

    printf("Safe buffer content: %s\n", buffer);

}


int main() {

    char user_input[256];


    printf("=== Buffer Overflow ===\n");

    printf("Enter text: ");

    fgets(user_input, sizeof(user_input), stdin);

    user_input[strcspn(user_input, "\n")] = 0;
```

```
    printf("\n--- Vulnerable Output ---\n");
    vulnerable_function(user_input);

    printf("\n--- Safe Output ---\n");
    safe_function(user_input);

    return 0;
}
```

**STEP 5:** Compiling the program with the stack protection disabled

Command used:

gcc -o TheBufferOverflow TheBufferOverflow.c -fno-stack-protector

Figure 8: compiling

**STEP 6:** Verification that the executable was created

Command used:

ls -la TheBufferOverflow

Figure 9: Verification successful

**STEP 7:** Testing the program to establish the normal state, check the results produced and enter text without exceeding the 16 characters coded.

Command used:

./TheBufferOverflow

Figure 11: Programming running and prompting user to enter text

Prompt- Enter text: Morning123

Figure 12: Results of entered text Morning123

RESULTS

=== Buffer Overflow ===

Buffer size: 16 characters

Enter text: Morning123

--- Vulnerable Function Output ---

Buffer content: Morning123

--- Safe Function Output ---

Safe buffer content: Morning123

=== Done ===

**STEP 8:** Triggering The Buffer Overflow by testing how the program will handle a long input when prompted to enter text.

Command used:

./TheBufferOverflow

Figure 13: Running program

Prompt- Enter text: AAAABBBBCCCCDDDDEEEEFFFFGGGG

Entering the above 28 characters will exceed the 16 characters of the buffer size causing a buffer overflow making the attack successful.

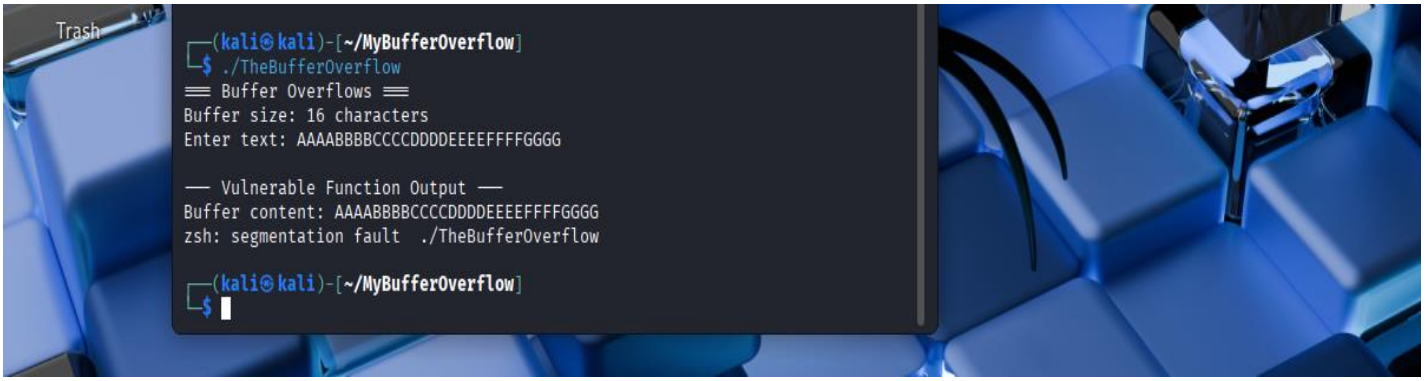


Figure 14: Buffer overflow successful

**RESULTS**

=== Buffer Overflows ===

Buffer size: 16 characters

Enter text: AAAABBBBCCCCDDDDEEEEFFFFGGGG

--- Vulnerable Function Output ---

Buffer content: AAAABBBBCCCCDDDDEEEEFFFFGGGG

25h: segmentation fault ./TheBufferOverflow

## HOW THE BUFFER FLOW PROGRAM WORKS

1. Buffer Created: characters buffer [16] makes space for 16 characters

2. Copy: strcpy() copies everything without checking size

3. Overflow: When input more than 16 characters, extra data overwrites other memory

4. The flag -fno-stack-protector disables compilers safety feature so we can demonstrate the actual vulnerability.

## ATTACK DIAGRAM

Buffer Overflow exceeding the buffer area, overflowing to base pointer and return memory corrupting the stack frame.

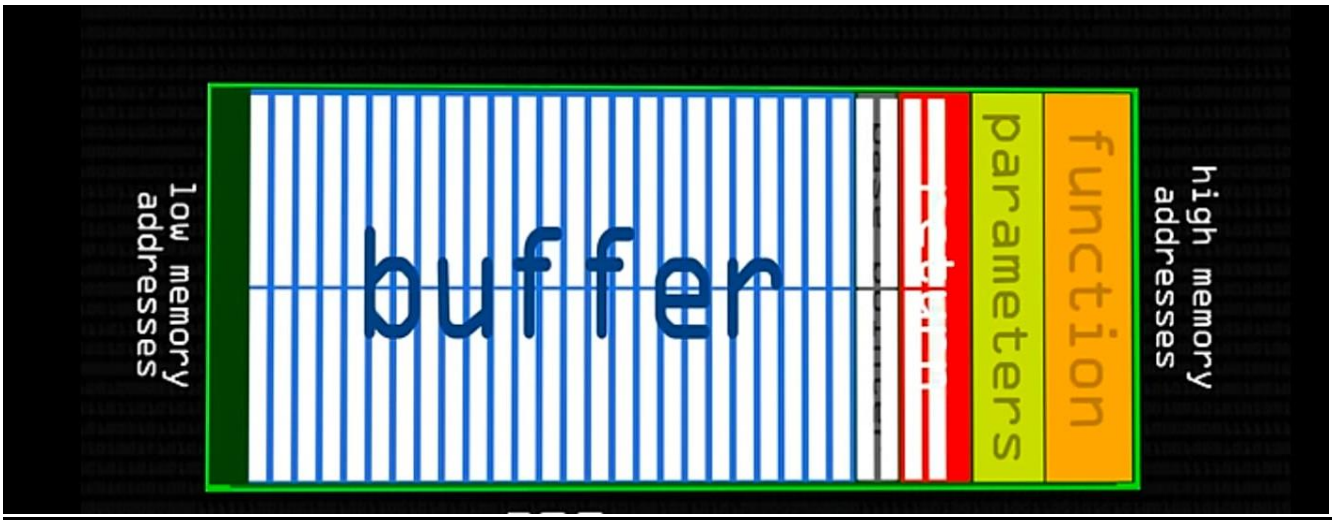


Figure 15: Buffer overflow on memory

Input: AAAABBBBCCCCDDDDEEEEFFFFGGGG (28 characters)

Explanation: The first 16 character's fill the buffer, the next character's overwrite frame pointer and the return address enabling a hacker to use this overflow to access the targets system and causing harm.

**CREATING THE ONE-CLICK SCRIPT-** This enables the program to run with one click making the buffer flow attack quick enabling the attacker to gain unauthorized access and cause harm to the system.

**Creating an automated script**

Command used:

nano click.sh

Figure 16: script

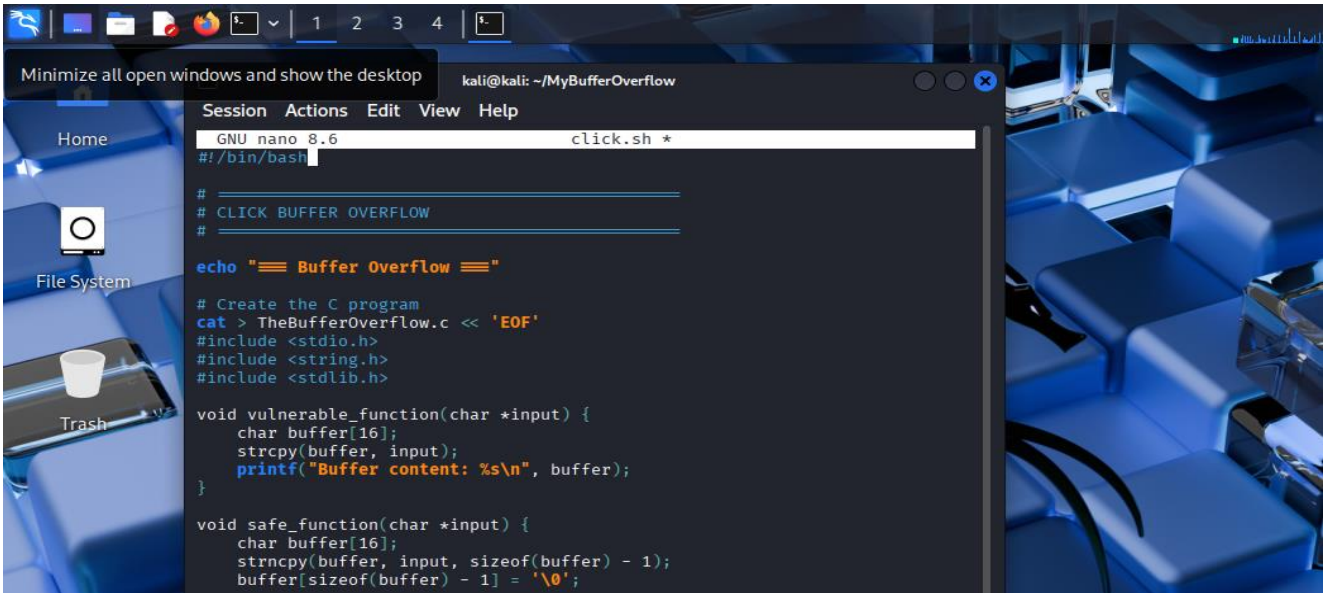


Figure 17: nano showing code click.sh

**Making it executable**

Command used:

chmod +x click.sh

Figure 18: executable

**Running script**

Command used:

./click.sh

**One-click results**

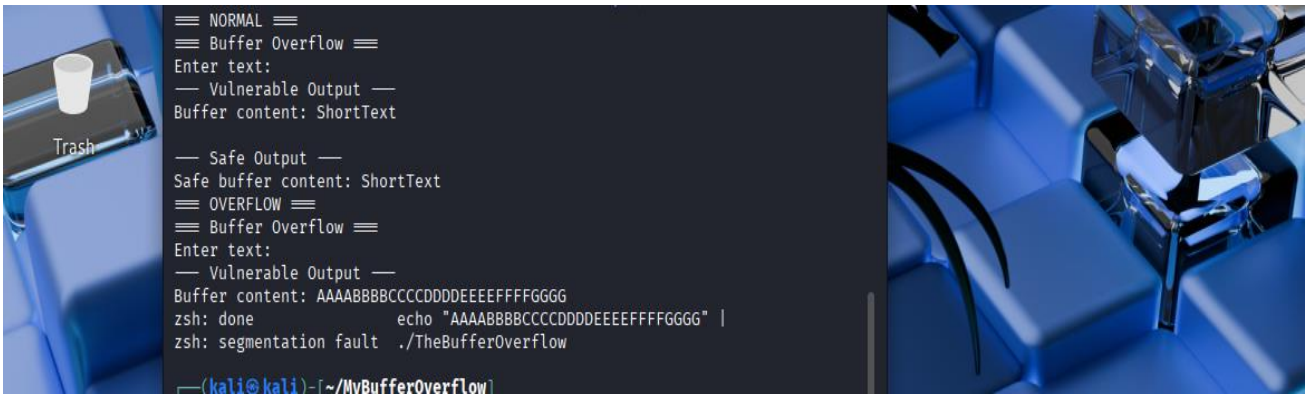


Figure 19: results

**Command to show everything**

Command used:

gcc -o TheBufferOverflow TheBufferOverflow.c -fno-stack-protector && echo "=== NORMAL ===" && echo "ShortText" | ./ TheBufferOverflow && echo "=== OVERFLOW ===" && echo "AAAABBBBCCCCDDDDEEEEFFFFGGGG" | ./ TheBufferOverflow

## HOW TO PREVENT BUFFER OVERFLOWS

Security measures into code development is essential by using programming languages that include built-in protection, regularly testing code to detect/fix errors and avoiding standard library functions that have not been bounds-checked, which includes gets, scanf, and strcpy.

Common runtime protection:

- **Address space layout randomization (ASLR):** ASLR moves at random around locations of data regions to randomize address spaces, which makes overflow attacks almost impossible.
- **Data execution prevention:** This method prevents an attack from being able to run code in non-executable regions by flagging areas of memory as executable or non-executable.
- **Structured exception handling overwrite protection (SEHOP):** SEHOP prevents attackers' malicious code from being able to attack the SEH and use its overwrite exploitation technique.

## CONCLUSION

The buffer overflow attack demonstrates the vulnerability attackers take advantage of causing damage to the system and gaining unauthorized entry. This milestone demonstrates the importance of proper input validation and secure coding practices in preventing memory corruption vulnerabilities that have been exploited by attackers.

## REFERENCES


Fortinet *Buffer Overflow*. Available at: https://www.fortinet.com/resources/cyberglossary/buffer-overflow

OWASP *Buffer Overflow*. Available at: https://owasp.org/www community/vulnerabilities/Buffer_Overflow

Imperva *Buffer Overflow*. Available at: https://www.imperva.com/learn/application-security/buffer-overflow/

*Figure 1: Example of Buffer Overflow Illustration* (2026) Available at: https://share.google/8jnFt6CXMN4gubzOC

Encypted guy *[*buffer overflowattack explained*]*. Available at: https://youtu.be/cHj4UkzcKwU

Navyaunnikrishnan9875 *[*buffer overflow attack*]*. Available at: https://youtu.be/njaQE8Q_Ems

Computerphile *[*running a buffer overflow atta*]*. Available at: https://youtu.be/1S0aBV-Waeo

Gamingwithmb3698 *[*buffer overflow tutorial in kali linux*]*. Available at: https://youtu.be/e7X6GaVMzPc

Apolloclark *[*buffer overflow attack*]*. Available at: https://youtu.be/eYrfWpkvMxA